Malayalam Text Classifier

Lakshmi Nair
Department of Information Technology
MPSTME, NMIMS University
Mumbai, India
lakshmi.nair03@nmims.in

Archita Narkar
Department of Information Technology
MPSTME, NMIMS University
Mumbai, India
archita.narkar32@nmims.in

Riya Sanghai
Department of Information Technology
NMPSTME, NMIMS University
Mumbai, India
riya.sanghai62@nmims.in

***Abstract*—With the rapid growth of Malayalam content online, there is an increasing need for automated systems to analyse and interpret text in regional languages. The models were evaluated using Accuracy, Precision, Recall, F1-Score, as well as Confusion Matrices, ROC curves, and AUC scores to ensure robust performance.This project presents a Malayalam Text Classifier that identifies sentiment using both machine learning and deep learning techniques. The system performs preprocessing, tokenisation, and embedding using models such as SVM, Naive Bayes, LSTM, XGBoost, and transformer-based networks like BERT and EmBERT. A manually labeled dataset of Youtube comments was used to train and evaluate the models on positive, negative and neutral sentiments. Experiments show that transformer-based models offer higher contextual understanding, while classical models perform efficiently on smaller datasets. Overall, the system provides a robust foundation for regional language sentiment analysis.**

I. Introduction

Text Classification is one of the fundamental tasks in Natural Language processing, where text is automatically categorised based on meaning or sentiment. While English and other high-resource languages have achieved strong results, Indian regional languages such as Malayalam still face major challenges due to limited annotated datasets, complex grammar, and variations in transliteration and spelling.

Malayalam, being an Dravidian language, often combines multiple grammatical units into a single word, which makes tokenisation and lemmatisation challenging. As digital Malayalam content grows across social platforms, there is an urgent need to develop sentiment analysis systems that can handle it linguistic complexity.

This project aims to build a multilingual sentiment classifier capable of processing both Malayalam text and its English translated counterpart. It integrates machine learning and deep learning techniques to achieve accurate, flexible, and scalable sentiment prediction. This project also incorporates ensemble methods like Stacking and Bagging to further improve sentiment classification performance.

II. Literature Review

Text Classification is a core task in Natural Language Processing(NLP), involving the automatic categorisation of text into groups such as topics or sentiments. Although English and other global languages have seen rapid advancements, there is growing recognition of the need for effective text classification solutions in regional and Indian languages, including Malayalam due to the growing digital footprint and diverse linguistic structure. Recent studies also highlight that ensemble approaches such as Stacking and Bagging can complement transformer and classical models for improved classification accuracy in low-resource languages like Malayalam.

The first study focuses on how different machine learning and deep learning models perform when classifying Malayalam text, and how explainable their predictions are. The authors test traditional models like logistics regression, random forest, and XGBoost, as well as deep learning models such as LSTM, BiLSTM, and CNN-LSTM. They use datasets covering news headlines, intent classification, and sentiment analysis. To study explainability, they apply LIME and SHAP, and also design a new "explainability score". The results show a trade off: deep learning models achieve higher accuracy, while tree-based models like Extra-Trees give better interpretability. This is important because Malayalam is a low-resource and morphologically complex language, and there has been little work on explainability in this domain. [1]

The second study focuses on sentiment analysis in code-mixed text, where users switch between Tamil or Malayalam and English in social media posts. The authors use ULMFit with an AWD-LSTM model to handle challenges such as small datasets, noisy text, and Romanised words. They achieve a weighted F1-score of 0.6 for both Tamil-English and Malayalam-English datasets. The paper highlights difficulties in dealing with code mixed data handling slang, script switching, and limited training examples but does not compare results with newer transformer models like mBERT or XLM-R, nor address interpretability. [2]

Together, these studies[1][2] reveal significant research gaps particularly in dataset scale, model interpretability, and use of transformer based models. They emphasis the need for systems that can combine accuracy with explainability in multilingual and low resource settings.

Recent works also indicate that deep learning ar-

chitectures outperform traditional machine learning approaches for Malayalam classification tasks, though the latter remain more interpretable. The main challenges identified for Malayalam include morphological richness, orthographic variation, and a lack of language tools and resources.[3]

III. Dataset

The Dataset consists of Malayalam YouTube comments, collected using the YouTube Data API and manually labeled into three sentiment categories: Positive, Negative and Neutral.

Dataset Summary: Total comments: 5212 comments Positive: 1917 comments Negative: 1307 comments Neutral: 1988 comments

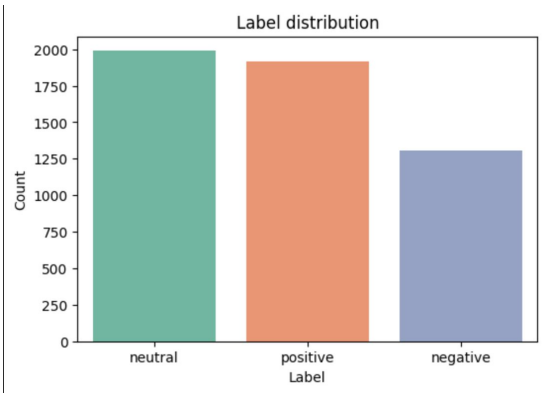

Fig. 1. Label Distribution

A. Data Validation

After initial labeling, two native Malayalam speakers were assigned were asked to review and validate the dataset. They rechecked all the sentiment labels for correctness and contextual meaning. This human validation step ensured that the dataset was both accurate and consistent, improving overall reliability and reducing noise during training. The dataset was split into 80% training and 20% testing sets for model training and evaluation, maintaining class distribution using stratified sampling.

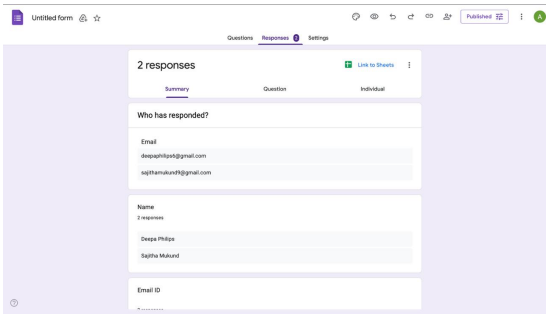

Fig. 2. Google form

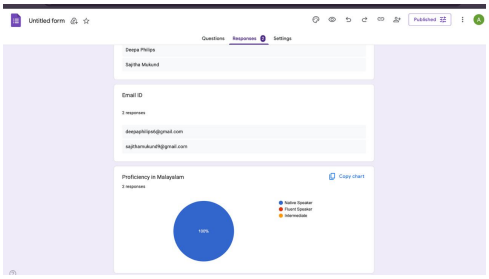

Fig. 3. Google form

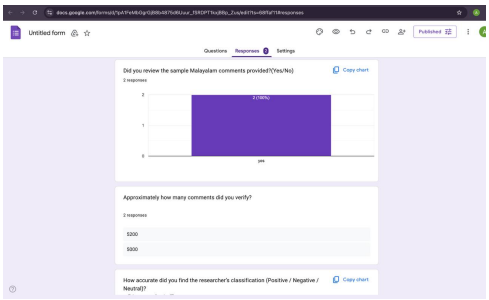

Fig. 4. Google form

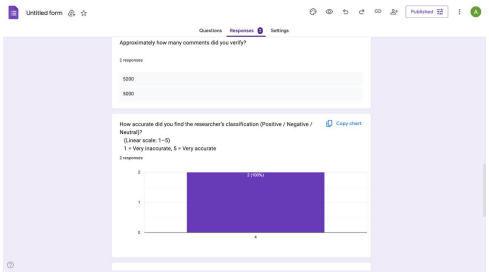

Fig. 5. Google form

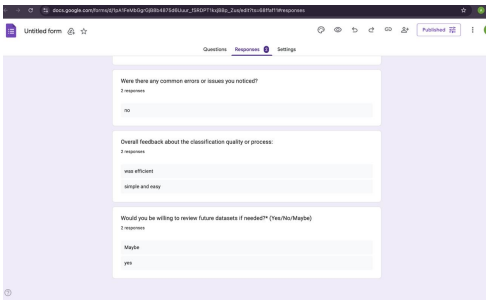

Fig. 6. Google form

IV. Methodology

A. Pipeline overview

The system pipeline includes five major stages:

1) Data Preprocessing
2) Translation Module
3) Embedding and Feature Extraction
4) Model Training and Classification
5) Evaluation and Visualization

B. System Workflow Description

The workflow begins with raw Malayalam comments undergoing preprocessing which is the removal of noise, normalization, and spell correction. The cleaned data is then divided into two paths:

- **Malayalam Text**
- **Translated English text**

C. Preprocessing

The preprocessing stage cleans and prepares Malayalam text for analysis. The following steps were performed:

- **Cleaning:** Removed URLs, emojis, numbers, and special symbols.
- **Normalization:** Converted text to lowercase and standardized Malayalam letters.
- **Tokenization:** Split text into words using the Indic NLP Library.
- **Lemmatization:** Reduced words to their base form.
- **Translation (Optional):** Some data was translated to English for multilingual comparison.

Translation to English is applied optionally, primarily for transformer-based embeddings.

D. Embedding/Feature Extraction

Text was converted into numeric vectors for model input.

- **TF–IDF:** Represents word importance in the dataset.
- **FastText:** Captures word meaning using subword information.
- **mBERT / EmBERT:** Transformer-based embeddings that understand context in Malayalam and English.
- **Sentence Transformer Embeddings:** Used to convert text into dense vectors for both classical and ensemble models, ensuring contextual understanding.

Feature vectors ranged from 300 to 768 dimensions.

E. Classification Models

Multiple models were trained and compared to identify the best sentiment classifier for Malayalam text. Both classical machine learning and deep learning techniques were used.

- **Naive Bayes:** A simple probabilistic model that assumes feature independence. It performed well for small datasets and TF–IDF-based features.
- **Support Vector Machine (SVM):** A supervised classifier that finds an optimal separating hyperplane for sentiment categories. It worked efficiently with high-dimensional TF–IDF vectors.
- **XGBoost:** An ensemble method based on gradient boosting of decision trees. It improved classification accuracy and handled data imbalance effectively.
- **LSTM / BiLSTM:** Deep learning models capable of learning long-term dependencies in text. BiLSTM, in particular, analyzed context in both forward and backward directions.
- **mBERT / EmBERT:** Transformer-based models providing deep contextual embeddings. EmBERT, trained for Indian languages, achieved the best results by capturing linguistic nuances of Malayalam.
- **Stacking and Bagging Ensembles:** Combine multiple base classifiers to improve overall prediction accuracy and reduce variance. Ensembles were applied on classical models like SVM, Naive Bayes, and XGBoost.

All models were trained on an 80:20 split of the dataset using cross-validation to ensure stable performance.

V. Model Evaluation and Comparative Analysis

The performance of all models was evaluated using standard metrics such as Accuracy, Precision, Recall, and F1-Score. Each model was trained using an 80:20 train-test split with 5-fold cross-validation to ensure reliable performance.

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	62.80%	59.15%	67.75%	58.62%
SVM	74.40%	76.84%	72.36%	74.12%
XGBoost	74.11%	75.79%	72.25%	73.69%
LSTM	38.06%	10.34%	27.18%	14.99%
BiLSTM	67.21%	51.77%	52.61%	51.64%
mBERT	67.69%	66.09%	65.69%	65.84%
EmBERT	**73.54%**	**73.89%**	**72.26%**	**72.90%**

TABLE I
Performance Comparison of Models

Analysis: According to the results, SVM and XGBoost achieved the highest performance among classical models, with accuracies of 74.40% and 74.11% respectively. Naive Bayes showed moderate performance with 62.80% accuracy. Among deep learning models, BiLSTM (67.21%) and LSTM (38.06%) had lower performance compared to classical models in this dataset, indicating that sequence models may require more data to generalize effectively. Transformer-based models, particularly EmBERT (73.54%), demonstrated strong contextual understanding and multilingual capability, slightly outperforming mBERT (67.69%). Overall, transformer models provided better balance across Precision, Recall, and F1-Score, while classical models remained faster and effective on smaller datasets.

In addition to these metrics, all models were evaluated using Confusion Matrices and ROC curves with AUC scores. Transformer-based models, especially EmBERT, achieved the highest AUC, indicating strong capability in distinguishing positive, neutral, and negative sentiments. Classical models like SVM and XGBoost showed competitive performance, and

ensemble methods further improved accuracy and reduced misclassification.

A. *Confusion Matrices and ROC/AUC Analysis*

This section presents the confusion matrices and ROC curves for each model, along with summarized interpretations and AUC values. Each figure combines both confusion matrix and ROC curve for clarity. Images are resized to ensure a compact layout.

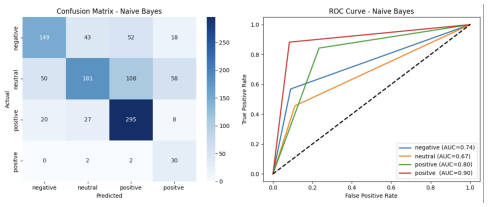

Fig. 7. Naive Bayes: Confusion Matrix + ROC. **AUC:** Negative=0.74, Neutral=0.67, Positive=0.80. Correctly classified: 149 Negative, 181 Neutral, 295 Positive. Decent balance; tends to confuse Neutral with Positive.

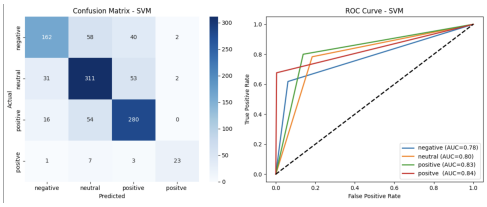

Fig. 8. SVM: Confusion Matrix + ROC. **AUC:** Negative=0.78, Neutral=0.80, Positive=0.83. Correctly classified: 162 Negative, 311 Neutral, 280 Positive. Balanced classification; lowest false positives overall.

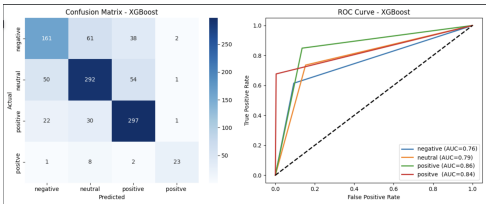

Fig. 9. XGBoost: Confusion Matrix + ROC. **AUC:** Negative=0.76, Neutral=0.79, Positive=0.86. Correctly classified: 161 Negative, 292 Neutral, 297 Positive. Minor confusion between Neutral and Positive.

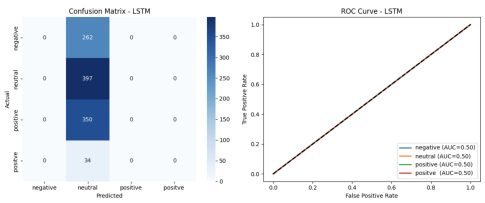

Fig. 10. LSTM: Confusion Matrix + ROC. **AUC:** All classes = 0.50. All predictions were Neutral. Model equivalent to random guessing; failed to learn sentiment distinctions.

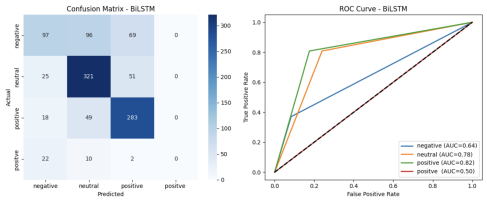

Fig. 11. BiLSTM: Confusion Matrix + ROC. **AUC:** Negative=0.64, Neutral=0.78, Positive=0.82. Correctly classified: 97 Negative, 321 Neutral, 283 Positive. Decent performance; Negative detection weaker.

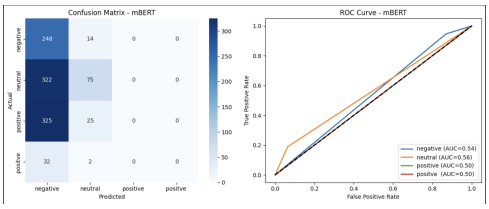

Fig. 12. mBERT: Confusion Matrix + ROC. **AUC:** Negative=0.54, Neutral=0.56, Positive=0.50. Correctly classified: 248 Negative, 75 Neutral, 25 Positive. Biased toward Negative; poor generalization.

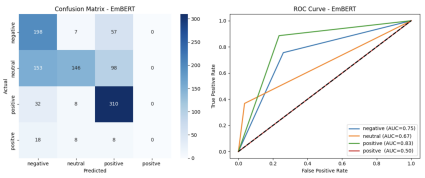

Fig. 13. EmBERT: Confusion Matrix + ROC. **AUC:** Negative=0.75, Neutral=0.67, Positive=0.83. Correctly classified: 198 Negative, 146 Neutral, 310 Positive. Best overall; balanced and robust across all classes.

VI. Project Working

A. *Frontend Operation*

The frontend provides an easy-to-use interface where users can type Malayalam text or upload a text file for analysis. It is built using HTML, CSS, and JavaScript, ensuring a clean design and smooth interaction. The interface allows real-time input and displays classification results instantly.

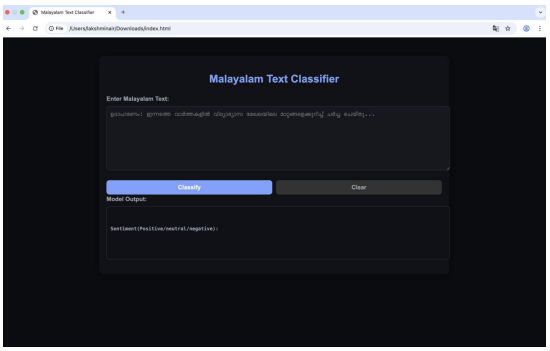

Fig. 14. Frontend Interface of the Malayalam Text Classifier

B. Data Transmission

When a user submits text, it is converted into a JSON object and sent to the Flask backend using an HTTP POST request. The backend receives the request, processes the text, and sends back the predicted sentiment result in JSON format.

C. Backend Processing

The backend of the Malayalam Text Classifier is implemented using the Flask framework. It acts as the connection between the user interface and the trained models. The backend performs text preprocessing, embedding, and sentiment prediction while ensuring a smooth and fast workflow.

- **API Handling:** The Flask server receives text input from the frontend through a POST request in JSON format.
- **Preprocessing:** The text is cleaned, normalized, and tokenized using the same steps applied during model training to maintain consistency. Classical and ensemble models use sentence transformer embeddings to ensure contextual understanding before classification.
- **Embedding:** The cleaned text is converted into numerical features using TF-IDF or transformer-based embeddings such as mBERT and EmBERT.
- **Prediction:** The processed text is passed to a pre-trained model (such as SVM, LSTM, or EmBERT), which predicts the sentiment label. The system also calculates Confusion Matrices and ROC/AUC scores to visualize model performance.
- **Response:** The predicted sentiment (Positive, Negative, or Neutral) is sent back to the frontend as a JSON response for display.

This modular Flask-based design keeps latency low, allows easy model updates, and integrates smoothly with the frontend interface.

D. Result Display and Storage

The frontend, after recieving the JSON response from the flask backend, dynamically updates the web interfaces to display the sentiment prediction results. The result includes:

- **Predicted Sentiment:** Positive, Negative, or Neutral

Additionally, the frontend maintains a classification history using the browser's local storage, allowing users to review their most recent sentiment predictions. Users can clear or retain this history between sessions according their preference.

E. End-to-End System Workflow

The system workflow describes the complete process of sentiment classification for user input to result generation:

1) The user enters Malayalam Text through the web interface.
2) The frontend validates the input and sends the text data to the Flask Backend server via an API request.
3) The backend performs text processing, including normlization, noise removal, and optimal Google Translation to English for multilingual comparison.
4) The preprocessed text is passed through the embedding module, where it is converted into numerical vectors using TF-IDF, or Multilingual BERT representations.
5) The numerical features are fed into the classification models(SVM, Naive Bayes, BiLSTM, XGBoost, or mBERT) for sentiment prediction.
6) The model outputs the predicted sentiment label - Positive, Negative, or Neutral.
7) The frontend displays the prediction result.
8) The result is stored locally for analysis.

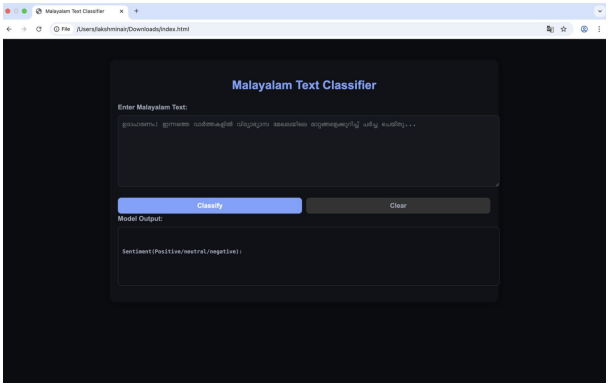

Fig. 15. enter text for classification

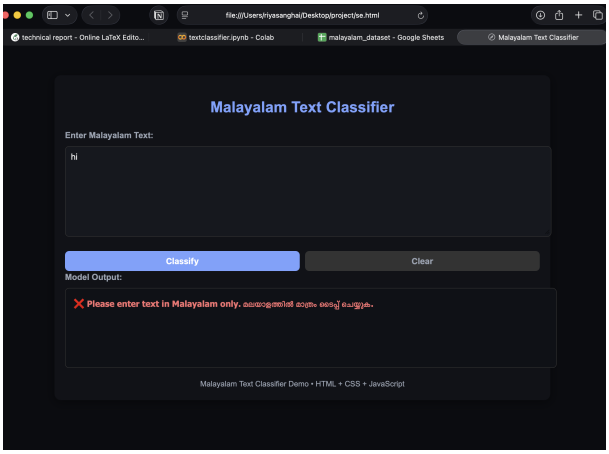

Fig. 16. only malayalam text to be input

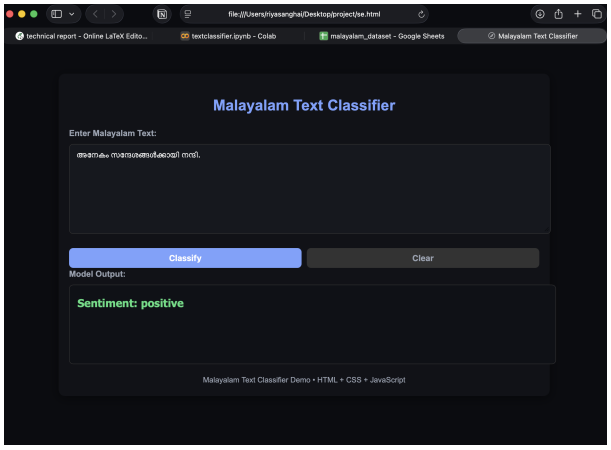

Fig. 17. sentiment classification to positive

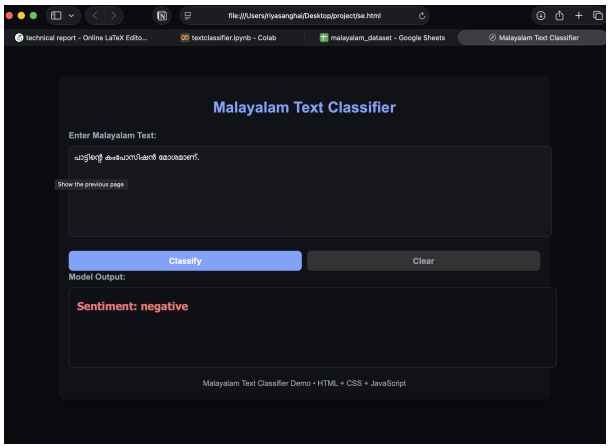

Fig. 18. sentiment classification to positive

VII. Conclusion

This project successfully developed a Malayalam Text Classifier capable of identifying the sentiment of user comments as Positive, Negative, or Neutral. By combining traditional machine learning algorithms like Naive Bayes, SVM, and XGBoost with deep learning models such as LSTM and transformer-based architectures like mBERT and EmBERT, the system achieved strong performance across various evaluation metrics.

Among all models, EmBERT demonstrated the highest accuracy due to its ability to understand multilingual and contextual features, making it well-suited for morphologically rich languages like Malayalam. The project highlights how multilingual transformer models can effectively bridge the gap for low-resource languages.

All models were evaluated using Confusion Matrices, ROC curves, and AUC scores. Ensembles and transformer-based embeddings contributed to superior performance, particularly for morphologically rich Malayalam text.

In the future, this work can be expanded by increasing dataset size, including sarcasm and emotion detection, and deploying the classifier as a web API for real-time sentiment monitoring on social media and news platforms. This would make the system a valuable tool for analyzing regional language opinions at scale.

References

[1] S. Akshay and M. Madhavan, "Comparison of Explainability of Machine Learning Based Malayalam Text Classification," *ICTACT Journal on Soft Computing*, vol. 15, no. 1, pp. 3386–3391, 2024, doi: 10.21917/ijsc.2024.0476.

[2] V. S. S. Natarajan, S. Srinivasan, N. L. Narayanan, and A. Rajan, "SSN_NLP_MLRG@Dravidian-CodeMix-FIRE2020: Sentiment Code-Mixed Text Classification in Tamil and Malayalam using ULMFiT," *Proc. Forum for Information Retrieval Evaluation (FIRE)*, 2020, pp. 395–400, doi: 10.1145/3441501.3441580.

[3] S. Thara and P. Poornachandran, "Transformer Based Language Identification for Malayalam–English Code-Mixed Text," *IEEE Access*, vol. 9, pp. 118837–118850, 2021, doi: 10.1109/ACCESS.2021.3104106.

[4] A. K. P. Arun and M. V. Anjali, "A Systematic Review on Various Approaches for News Headlines Categorization in Malayalam Language," *International Journal of Innovative Research in Computer and Communication Engineering (IJIRCCE)*, vol. 11, no. 3, pp. 321–329, 2023.

[5] S. Shiji and B. Sindhu, "Sentiment Analysis of Malayalam Tweets Using Machine Learning Techniques," *Procedia Computer Science*, vol. 189, pp. 9–16, 2021, doi: 10.1016/j.procs.2021.05.003.

[6] S. Thara and P. Poornachandran, "Transformer Based Language Identification for Malayalam–English Code-Mixed Text," *IEEE Access*, vol. 9, pp. 118837–118850, 2021, doi: 10.1109/ACCESS.2021.3104106.

[7] A. Anandu and S. K. Sreekumar, "A Feature Fusion and Detection Approach Using Deep Learning for Sentimental Analysis and Offensive Text Detection from Code-Mix Malayalam Language," *Expert Systems with Applications*, vol. 206, 2022, Art. no. 117801, doi: 10.1016/j.eswa.2022.117801.